Full-Stack Build Roadmap

Prepared for: Manas Jiwnani | Target: Williams-Sonoma Campus Drive | MIT WPU, Pune | May 2026

Swift	**SwiftUI**	**UIKit**	**Node.js**	**Express**
Supabase	**PostgreSQL**	**REST API**	**Git**	**Xcode**

What You'll Build

A full-stack iOS eCommerce app inspired by Williams-Sonoma — with a custom Node.js + Supabase backend, SwiftUI frontend, real REST API integration, authentication, and live deployment. Everything you need to walk into the interview and say: I built this.

Phase	Focus	Duration	Output
1	Setup & Architecture	1 day	Repo, Supabase DB, folder structure
2	Node.js Backend	3–4 days	Live REST API with 6 endpoints
3	iOS App — Core UI	4–5 days	Product list, detail, search screens
4	iOS App — Advanced	3–4 days	Cart, auth, animations
5	Polish & Deploy	2 days	Deployed backend, GitHub README
6	Interview Prep	1 day	Talking points, demo script

1 Setup & Architecture

Day 1 | Foundation before you write a single line of feature code

1.1 Create Your GitHub Repository

- ■ Go to github.com → New Repository → name it **ws-ecommerce-ios**
- ■ Add a README.md immediately (describe the project in 3 lines)
- ■ Create two folders in the repo: **/backend** and **/iOS**
- ■ Set up **.gitignore** for Node.js and Xcode (use gitignore.io)
- ■ Pin this repo on your GitHub profile — recruiters look at pinned repos

1.2 Set Up Supabase Database

Go to supabase.com → New Project → name it **ws-store**. Then create these tables in the SQL editor:

Table	Key Columns	Purpose
products	id, name, price, description, image_url, category, stock	Product catalogue
users	id, email, name, created_at	Customer accounts
cart_items	id, user_id, product_id, quantity	Shopping cart
orders	id, user_id, total, status, created_at	Order history

✓ Resume Impact: Supabase goes from a floating keyword to a real, described project asset.

1.3 Backend Folder Structure

```
backend/ ■■■ server.js (entry point) ■■■ routes/ ■ ■■■ products.js ■ ■■■ cart.js ■ ■■■ orders.js
■■■ middleware/ ■ ■■■ auth.js ■■■ .env (Supabase keys — never commit this) ■■■ package.json
```

2 Node.js + Express Backend

Days 2–5 | Build the REST API your iOS app will talk to

2.1 Initialize the Project

```
cd backend && npm init -y && npm install express @supabase/supabase-js dotenv cors
```

2.2 API Endpoints to Build

These 8 endpoints give you a complete, interview-ready backend:

Method	Endpoint	What it does	Interview Value
GET	/products	Fetch all products	Core REST concept
GET	/products/:id	Fetch single product	URL params
GET	/products?category=	Filter by category	Query params
POST	/cart	Add item to cart	POST body handling
GET	/cart/:userId	Get user's cart	Relational data
DELETE	/cart/:itemId	Remove from cart	DELETE method
POST	/orders	Place an order	Business logic
GET	/orders/:userId	Order history	Data aggregation

2.3 Sample Route Code (products.js)

```
const express = require('express') const router = express.Router() const { supabase } =
require('../supabaseClient') // GET all products router.get('/', async (req, res) => { const {
category } = req.query let query = supabase.from('products').select('*') if (category) query =
query.eq('category', category) const { data, error } = await query if (error) return
res.status(500).json({ error }) res.json(data) }) module.exports = router
```

2.4 Deploy the Backend (Free)

- Push backend/ to GitHub
- Go to **railway.app** → Deploy from GitHub → select your repo → set /backend as root
- Add environment variables: SUPABASE_URL and SUPABASE_ANON_KEY
- Railway gives you a live URL like **https://ws-store.up.railway.app**
- Test every endpoint in **Postman** before connecting iOS — screenshot these for your portfolio

✓ Resume Impact: 'Built and deployed a REST API with Node.js, Express, and Supabase — handling product catalogue, cart management, and order processing.'

3 iOS App — Core UI

Days 6–10 | SwiftUI screens that talk to your real backend

3.1 Xcode Project Setup

- Open Xcode → New Project → App → Interface: **SwiftUI** → Language: **Swift**
- Name it **WSStore**
- Add folder groups: Views/, Models/, Services/, Components/
- Create **APIService.swift** — this will handle all network calls using URLSession

3.2 App Architecture — MVVM

```
Use MVVM (Model-View-ViewModel) — this is industry standard and a great interview talking point: Model: Product.swift, CartItem.swift, Order.swift (Codable structs) ViewModel: ProductViewModel.swift (@Published properties, API calls) View: ProductListView, ProductDetailView, CartView (SwiftUI)
```

3.3 Screens to Build

Screen	Key Features	Skills Demonstrated
Product List	Grid layout, image loading, pull-to-refresh, category filter tabs	LazyVGrid, AsyncImage, @StateObject
Product Detail	Full image, price, description, Add to Cart button, stock badge	NavigationLink, custom modifiers
Search	Real-time search bar filtering results from API	Combine, debounce, @Published
Cart	List of items, quantity stepper, subtotal, Place Order button	List, Stepper, computed properties
Order History	Past orders with status badge (Processing/Shipped/Delivered)	Enum, conditional styling
Splash / Login	Animated logo, email login screen (even if mock)	Animation, onAppear, @AppStorage

3.4 Connecting iOS to Your Backend (APIService.swift)

```
struct Product: Codable { let id: Int let name: String let price: Double let imageUrl: String let category: String } class APIService { static let baseURL = "https://ws-store.up.railway.app" func fetchProducts() async throws -> [Product] { let url = URL(string: "\(baseURL)/products")! let (data, _) = try await URLSession.shared.data(from: url) return try JSONDecoder().decode([Product].self, from: data) } }
```

✓ Resume Impact: 'Implemented MVVM architecture in SwiftUI with async/await URLSession for REST API integration — fetching real product data from a custom Node.js backend.'

4 Advanced Features

Days 11–14 | Features that separate you from other candidates

4.1 Features That Maximize Resume + Interview Value

Feature	Why It Matters	Difficulty
Offline Caching (UserDefaults)	Shows you think about real-world app behaviour	**Easy**
Skeleton Loading Screens	Professional UX — interviewers notice polish	**Easy**
Category Filter (tab bar)	Matches WS's actual product browsing pattern	**Easy**
Haptic Feedback on Add-to-Cart	Attention to iOS-specific UX details	**Easy**
Async Image with placeholder	Standard production pattern for image loading	**Easy**
Search with debounce (Combine)	Shows Combine framework knowledge	**Medium**
Cart persistence across sessions	Real product thinking — cart survives app restart	**Medium**
Custom animations (spring, easing)	Demonstrates SwiftUI animation fluency	**Medium**
Error states + retry button	Production-grade thinking — not just happy path	**Medium**
Dark Mode support	One modifier, huge professionalism signal	**Easy**

4.2 The One Feature That Will Make Them Remember You

Add a **"Recommended for You"** section on the Product List screen. When a user views a product, log the category. On the home screen, show 3 products from their most viewed category at the top. This is exactly what Williams-Sonoma's eCommerce AI team works on. In the interview, say: 'I built a basic recommendation engine based on browsing history — which I understand is something your team works on at scale with ML.' That's the moment.

5 Polish & Deploy

Days 15–16 | Make it look like a real product, not a student project

5.1 GitHub README (This is your Portfolio Page)

Your README must include:

- **Project banner image** — screenshot your app, put it at the top
- **Tech stack badges** — shields.io badges for Swift, Node.js, Supabase
- **Features list** with checkmarks
- **Architecture diagram** — simple box diagram: iOS App → Node.js API → Supabase
- **API documentation** — table of all 8 endpoints with method, URL, description
- **Setup instructions** — how to run backend locally and open Xcode project
- **Demo GIF or video link** — record your screen, upload to YouTube, link it

5.2 Add to Your Resume

Replace your current project description with this:

> **WSStore — iOS eCommerce App | github.com/manasjiw26/ws-ecommerce-ios**
>
> Built a full-stack eCommerce iOS application with a SwiftUI frontend and custom Node.js + Express REST API backend.
> Integrated Supabase (PostgreSQL) for product catalogue, cart management, and order processing.
> Implemented MVVM architecture, async/await networking, search with Combine, and offline caching.
> Deployed backend on Railway with 8 live API endpoints; app supports dark mode and custom animations.
> **Technologies:** Swift, SwiftUI, UIKit, Node.js, Express, Supabase, REST APIs, Git, Xcode

5.3 Quick Pre-Interview Fixes to Your Resume

Fix	Action
Typo in Skills	Change 'Databaes' → 'Databases' immediately
Missing SwiftUI	Add SwiftUI to your Skills → Frameworks section
Missing REST APIs	Add 'REST APIs, JSON' to Skills → Frameworks
No GitHub link	Add github.com/manasjiw26 to your header contacts
Floating Supabase	Now tied to this project — add the GitHub link next to it

6 Interview Prep

Day 17 | What to say when they ask about your project

6.1 Technical Test — What to Expect

Williams-Sonoma's technical test will likely cover:

- ■ **Swift basics** — optionals, closures, protocols, structs vs classes
- ■ **OOP concepts** — inheritance, encapsulation, polymorphism
- ■ **iOS fundamentals** — UIKit lifecycle, SwiftUI state management (@State, @Binding, @ObservedObject)
- ■ **REST API questions** — what is a GET vs POST, what is JSON, how do you decode it in Swift
- ■ **DSA basics** — arrays, dictionaries, basic sorting (your 170+ GFG problems help here)
- ■ **Git** — what is a branch, how do you resolve merge conflicts

6.2 Interview Talking Points — Project Demo Script

They Ask	You Say
Tell me about a project you built	I built a full-stack iOS eCommerce app — SwiftUI frontend, Node.js REST API, Supabase database. I designe
Have you worked with REST APIs?	Yes — I built the API myself. I know GET, POST, DELETE methods, query params, URL params, JSON enco
Why did you choose Supabase?	eCommerce data is relational — products, orders, cart items all reference each other. Supabase uses Postgre
Do you know MVVM?	Yes — I used it in this project. Models are Codable structs, ViewModels hold @Published state and call the AP
What would you improve?	I'd add user authentication with JWT, push notifications for order updates, and a proper recommendation syste

Final Submission Checklist

- ■ GitHub repo is public with a complete README and demo GIF
- ■ Backend is deployed and all 8 endpoints return real data
- ■ iOS app runs on simulator without crashes
- ■ Resume updated: typo fixed, SwiftUI added, REST APIs added, GitHub link added
- ■ Project description on resume links to GitHub repo
- ■ You can explain every line of your code in the interview
- ■ You've tested the app on both light and dark mode
- ■ Registered before May 4th 12 PM deadline

Good luck, Manas. You already have the iOS internship at Infosys, the CGPA, and the right skills. This project is the story that ties everything together.